# Universita' degli Studi di Pavia

Bachelor degree in Artificial Intelligence

# Cannon Game – Final exam project

# A. Y. 2023 / 24

**Lorenzo Brusati_535355 e Tsuhuy Ecaterina_535601**




***Computer programming, Algorithms and Data structure, Mod.1***

# Contents


**Cover**.....1
**Contents**..... **2**
**1. Introduction**.....**4**
**2. Goal of the project**.....**4**
2.1 Problem to be solved..... 4
2.2 Constraints..... 5
**3. Assumptions, methods, procedures**.....**6**
3.1 Assumptions..... 6
3.2 Game world and mechanics..... 8
Game world..... 8
Game mechanics:..... 9
3.3 Player and Entities..... 10
Player:..... 11
Entities:.....15
**4. Logical Structure**..... **19**
**5. Programming Implementation**..... **20**
5.1 Menu..... 20
5.1.1 Kivy Design..... 20
5.1.2 Play Button..... 20
5.1.3 Leaderboard, Help, Quit.....21
5.2 Save and Load screen.....21
5.2.1 Kivy design.....22
5.2.3 Nickname Input and Submission.....22
5.3 Selection screen..... 23
5.3.1 Kivy design.....23
5.3.2 Tank and Cannon Selection.....24
5.4 Levels screen.....24
5.4.1 Kivy design.....25
5.4.2 Levels specifications..... 25
5.4.3 Level 1 as example.....26
5.4.3.1 Kivy design..... 26

5.4.3.2 Gameplay mechanics..... 27
5.5 Leaderboard..... 28
5.5.1 Kivy design..... 28
5.5.2 Player data loading and Display..... 29
5.4.3 Dynamic Updates..... 29
5.6 Help..... 29
5.6.1 Kivy design..... 30
5.5.2 Loading Help Text..... 30
5.5.3 Text Formatting..... 31
5.5.4 User Experience..... 31
**6. Possible Improvements..... 32**
**7. Tools and Resources..... 33**
7.1 Libraries used..... 33
7.2 References..... 33

# 1. Introduction

The **Cannon** game is a 2D side-view artillery game, developed as part of the final exam project for the course on *Computer Programming, Algorithms, and Data Structures*. The project is inspired by classic artillery games and was designed using the **Kivy framework** in Python. The goal of the game is to control a cannon, adjust its angle and fire various types of projectiles to hit a target.

In each level, the player must fire a limited number of shots and can choose different types of projectiles, each with unique physical properties and behaviors.

# 2. Goal of the project

The primary goal of this project is to design and implement a 2D artillery game, inspired by classic games from this same genre, using the Kivy framework in Python. The project focuses on several key objectives:

## 2.1 Problem to be solved

The project aims to address a series of game development challenges, including:

- Implementing realistic projectile motion
- Implementing the correct shooting position of the cannon
- Create a smooth cluster bomb animation
- Creating an engaging gameplay experience

- Dynamic environment interaction: design obstacles that interact differently with various projectiles, such as reflecting lasers or being destroyed by projectiles.
- Think about the logical level structure and assignment of scores, deciding between different levels or only 1 endless game and how to collect values/points.
- Adaptive gameplay based on user input: implement a system that allows for smooth adjustments to the canon's angle and projectile choice, ensuring precision in gameplay mechanics.
- Find a way to manipulate player data in order to save and load them and show the score in a leaderboard.

### 2.2 Constraints

There as been many significant constraints given by the project's instructions:

- Strictly specifications about projectiles, obstacles, bullet dynamics, and game functionalities.
- Real-time physics calculations: since the game involves projectile motion, collisions and object interactions, real-time calculations had to be optimized so the game would run efficiently.
- Limited flexibility of Kivy for game development: while powerful for app development, is less suited for real-time video game mechanics. A significant challenge was optimizing the framework's capabilities to handle continuous physics calculation, animations and dynamic object interactions.

# 3. Assumptions, methods, procedures

## 3.1 Assumptions

Several assumptions were made during the development of Cannon game:

- Fixed window size: the game operates within a predefined window size, ensuring consistent gameplay and visual scaling.
- Keyboard input: the player uses keyboard inputs to control the cannon's movement and actions, trying to follow intuitive game keys and spacebar. The 'a' and 'd' keys are used to adjust the cannon's firing angle, providing precise control over the shot's direction.

```
            if "a" in self.keysPressed:
                new_angle += angle_change
            if "d" in self.keysPressed:
                new_angle -= angle_change
```

The '**i**', '**o**' and '**p**' keys allow the player to choose between different types of projectile: bullet, bombshell and laser, respectively. The spacebar is used to fire the chosen projectile.

```
    def _on_key_down(self, keyboard, keycode, text, modifiers):
        # print(f"Key down: {keycode[1]}")
        key = keycode[1]
        if key == 'spacebar':
            if self.cooldown == 0:
                self.fire()
```

```
            self.cooldown = 1
            Clock.schedule_once(self.update_cooldown, 1)
        # KEYS FOR BULLETS
        elif key == 'i':
            self.current_bullet = 'bullet'
        elif key == 'o':
            self.current_bullet = 'bombshell'
        elif key == 'p':
            self.current_bullet = 'laser'

        # KEYS FOR MOVEMENT
        elif key == 'd':
            self.keysPressed.add("d")
        elif key == 'a':
            self.keysPressed.add("a")
        return True
```

By default, the projectile is set to bullet unless the player switches to another type using the designed keys.

- Desktop environment: the game is designed for desktop computers, assuming the availability of keyboard and mouse input, without optimization for mobile devices or other platforms.

- Interaction between projectiles and entities: the interaction assumes that all projectiles behave consistently when colliding with entities.
- Single-player focus: the game is designed for a single player, with no multiplayer or networked gameplay functionalities.

### 3.2 Game world and mechanics

**Game world**

The cannon game takes place in a world where the player's city has come under attack by a hostile enemy force. The player assumes the role of a tank operator defending their city. The game is divided into multiple levels that reflect the player's journey from training to full-scale war, culminating in a shocking revelation about the true nature of the enemy.

- *Lore of the game*:

  The player begins with basic training in the first two levels, honing their skills in tank operation and target shooting. These levels serve as a preparation for the real conflict. By the third level, the player is thrust into a real war, battling the enemy forces that threaten their city. Initially, the player believes it to be a typical war, but as the game progresses, they uncover the truth: the city's enemies are not just humans, but robots orchestrating the attacks. This twist is revealed in the final level, changing the stakes and giving the player a new understanding of the conflict.

**Game mechanics:**

The core mechanics of the game revolve around projectile control and target aiming, with additional challenges created by the interaction between projectiles and obstacles.

The player during the game can select from different projectile types (e.g. bullet, bombshells, lasers), each with unique property:

- Bullets follow a parabolic trajectory and are affected by gravity
- Bombshells behave similarly to bullets, but it behaves like a cluster bomb.
- Lasers follow a straight line and are unaffected by gravity, but can reflect off certain obstacles.

Obstacles in the environment are located in such a way it's challenging to hit a specific target/s, depending on the current level of the game. This requires the player to experiment with different projectiles and adjust their aiming strategy accordingly.

In some levels, enemies appear as dynamic elements that move across the screen and they may also have specific health points or behaviors, such as exploding upon impact.

The game ends when the player eliminates the main target or when he exhausts its supply of ammunition since, in every level, ammo limits are imposed.

When the player successfully hits an entity, they receive immediate feedback through visual effects (such as explosions) and sounds (such as the sound of a collision).

Also the simplified physics system ensures that projectiles behave predictably, but still allow for enough complexity to create interesting interactions between entities and obstacles.

The player's main goal is to reach the last level in which he has to defend as much as possible the city until the main boss spawn and eliminate it. Based on his performance in the last level, the score will be updated once it will do better than his previous time, if he retry the game.

Here there is a brief recap on each level gameplay structure:

- **Level 1**: The player has to hit all the 9 training target and get confidence with the different types of bullets
- **Level 2**: The player has to hit the training crates that falls from the sky, if the crates hit the ground, the player loses points
- **Level 3**: the player has to hit enemies located above his head trying to eliminate them with combination of lasers and glasses
- **Level 4**: the player has to eliminate the massively-protected Capitan, depleting his lifebar.
- **Level 5**: the final battle: the player has to hit the final Boss that spawn after 20 seconds, paying attention to the bombs dropping from the airplane and if they hit the ground, the player loses points.

### 3.3 Player and Entities

In the Cannon game, a variety of entities, obstacles, and enemies are used to create a dynamic game environment. The player interacts with these entities through actions such as shooting and avoiding obstacles.

**Player:**

The player controls a cannon that fires projectiles to eliminate enemies and navigate through various levels. The player's primary interaction is through shooting bullets that can destroy enemies, by also rotating the cannon which directly impacts the trajectory of the projectiles, or interact with obstacles like glass.

The instance of the cannon is composed of two separate classes: tank and part_tank, referring respectively to the stationary part of the cannon and the moving one. Most of the functions related to the game are associated with the second class. here there's a brief recap of the respective functions:

**Tank Class:**

- __init__(self, **kwargs): Initializes the tank's visual representation as a `Rectangle` on the canvas, ensuring it updates with changes in position or size.
- update_rect(self, *args): Keeps the tank's graphical representation synchronized with its widget's position and size.
- update_canvas(self): Redraws the canvas when necessary, particularly useful when changing the tank's appearance.

**Part Tank Class:**

- __init__(self, **kwargs): Sets up the tank's movement, rotation, projectile firing, and sound effects, initializing the state and key properties.
- move_and_update_canvas(self, dt): Updates the tank's movement and redraws it in the game loop based on user input and time elapsed.

- move_angle(self, dt): Handles tank rotation, adjusting the angle based on key presses (`a` or `d`), with angle clamping for control.
- fire(self): Fires a projectile based on the current selected bullet type (bullet, bombshell, or laser) and calculates the firing position. and wuìith the functions fire_bullet(), fire_bombshell(), fire_laser() fires the respective projectile, adjusting speed and position, and reducing ammunition.
- explosion(self): Plays an explosion animation and sound when a projectile is fired, scheduling frames at regular intervals.
- _on_key_down(self, keyboard, keycode, text, modifiers) , _on_key_up(self, keyboard, keycode): Manages key press and release events for firing, movement, and switching bullet types.

Firing function:

```
    def fire(self):

# Handles firing a projectile, adjusting based on the current bullet type

        if not self.pause:

            angle_rad = radians(self.rotation_angle)

            d = sqrt(self.width**2 + (1/2) * self.height**2)

            theta = 0  # Default angle

            if self.current_bullet == 'laser':

                theta = arccos(self.width / d)

            self.cannon_x = self.x + int(d * cos(angle_rad + theta))

            self.cannon_y = self.y + int(d * sin(angle_rad + theta))

            if self.current_bullet == 'laser':

                self.fire_laser()

            if self.current_bullet == 'bullet':

                self.fire_bullet()

            if self.current_bullet == 'bombshell':

                self.fire_bombshell()
```

Firing function: example of the specific bullet shoot:

```
def fire_bullet(self):

        # Fires a standard bullet with velocity based on tank's rotation

        bullet_velocity = 500

        angle_rad = radians(self.rotation_angle)

        velocity_x = bullet_velocity * cos(angle_rad)

        velocity_y = bullet_velocity * sin(angle_rad)



        if self.munizioni > 0:

            # Trigger explosion effect and launch the bullet

            self.explosion()

            bullet = Bullet(pos=(self.cannon_x, self.cannon_y),
x_velocity=velocity_x, y_velocity=velocity_y)

            self.parent.add_widget(bullet)

            self.parent.projectiles.append(bullet)

            self.munizioni -= 1  # Decrease ammunition count

        else:

            print('munizioni terminate')  # Output message if out of ammo
```

**Entities:**

Entities are the main elements for which the projectiles can collide with. The entities inherit from a base class `Entities`, and each entity subclass has specialized behaviors, properties and canvas that are tailored to its role in the game. Below is a detailed breakdown of the structure and functions associated with each subclass. The game divides its entities into obstacles, enemies, and collectable items, each with specialized behavior:

- Obstacles are primarily static or moving blocks.

- Enemies vary in behavior, with some falling from the sky and others acting as bosses with health and explosion effects.

- Collectables increase the player's resources, such as ammunition.

Glass is a class “a part” that has particular interactions depending on the type of projectile.

Entities include attributes like position and size and have methods for movement and collision detection. In particular, the `check_collision` method ensures that when a bullet hits an entity, the appropriate action is triggered, such as damage to an enemy or reflection by obstacles like glass.

Here as follows there are presented all the Entities subclasses:

- **Glass:** reflects only laser projectiles and represents a type of indestructible obstacle.

```
    def on_collision(self, bullet):

        if bullet.__class__.__name__ == "Laser":
```

```
        if self.size[0] > self.size[1]:
            bullet.reflect_laser2()
        else:
            bullet.reflect_laser1()
```

- **Obstacle:** a category that includes static and dynamic objects like planes, blocks, and terrain, which obstruct the player's shots or movement since they cannot be destroyed by projectiles. Simplest interactions are allowed. Its subclasses are:
    - Plane: Represents a static plane obstacle. It uses a specific image.
    - Ground: Represents the ground, which also uses a static image.
    - Block_horiz and Block_horiz_2: These represent horizontally moving blocks. They adjust their velocity and direction when hitting the screen edges.
    - Block_vert and block vert 2: A vertically moving block with similar movement logic as horizontal blocks.
    - Jet: A vertically moving jetpack entity, changing direction upon reaching specific height thresholds.

- **Enemy:** hostile characters the player must defeat using projectiles, each with unique behaviors such as falling from the sky or requiring multiple hits to be destroyed, and unique values that modifies the score.

The enemy subclasses include:

- Sky_enemy: Represents enemies falling from the sky. It has a defined velocity and interacts with terrain.
- Training_falling: A smaller variant of `Sky_enemy`, using a different image.
- Capitan: An enemy with more health (life = 5). It reduces life points on collision and updates a life bar.
- ILBOSS: A boss enemy with more health and a higher point value. It triggers an explosion and reduces the boss's life upon collision.
- Training_Enemy, Soldier1, Soldier2, and Soldier3: Variations of enemy soldiers with different graphics, each having a point value of 1.

```
    def on_collision(self, bullet):

        self.create_explosion()

        self.explosion_sound.play()



        fade_anim = Animation(opacity = 0, duration = 0.2)

        fade_anim.bind(on_complete = lambda *args: self.remove_enemy())

        fade_anim.start(self)
```

The enemy class is further subdivided into various enemies types, each with distinct characteristics: these include enemies like `Training_falling` and

`Sky_enemy`, which follow different movement patterns and require varying strategies to defeat.

While most enemies trigger explosions and disappear upon impact, the `Capitan` class introduces additional complexity, the player has to hit him 5 times:

```
    def on_collision(self, bullet):

        self.life -= 1

        self.parent.ids['life'].update_life_bar(self.life)

        self.explosion_sound.play()

        if self.life == 0:

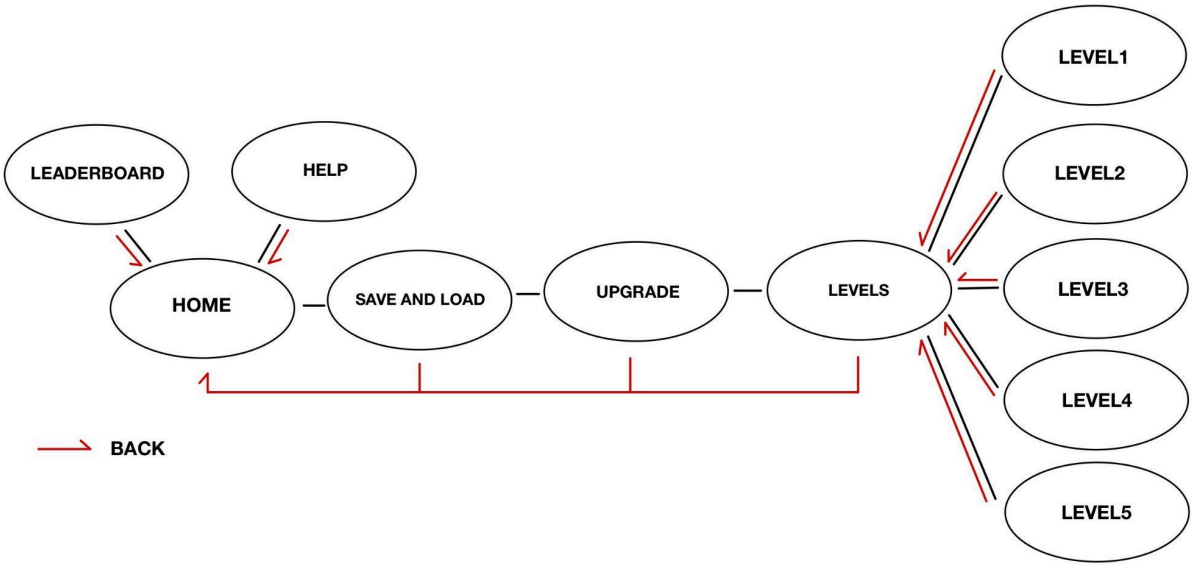
            self.remove_enemy()
```

- **Collectables:** objects like ammo drops that the player can collect to increase resources, such as ammunition. When collected, the player's ammunition count is increased by 5 and removes the collectable item from the game after it is collected.

  Subclasses include AmmoDrop (represents a stationary ammunition drop with its own graphic) and AmmoDrop_faling (a falling variant which moves downwards with a defined velocity)

Note that each subclass has its own behavior for handling collisions. For example,  enemies trigger explosions upon impact, while obstacles might reflect or block projectiles, depending on their type.

# 4. Logical Structure




*Navigation Structure for the Kivy-Based Game*

The following diagram illustrates the hierarchical structure and navigation flow of the user interface (UI) screens in the developed Kivy-based game application. Each node represents a distinct screen within the application, while the connecting lines symbolize the transitions between these screens, triggered by specific user actions, such as button presses. To ensure that each screen cloud be refreshed, at each change screen event it has been developed in the screen Manager the function goto_screen(name): it lets us refresh the previous instantiation of screen and add the required one via the variable ‘name’. Red arrows in the diagram represent the "back" navigation functionality, indicating the ability to return to the Home screen or a prior screen. This backward navigation ensures that the user can always return to previous sections of the game without being locked into one specific screen.

# 5. Programming Implementation

## 5.1 Menu

### 5.1.1 Kivy Design

The main menu screen is designed using Kivy's ***GridLayout*** and FloatLayout.

The screen is divided into two sections: the top displays the game title "Cannon Game," while the bottom section contains four buttons for navigation.

The ***GridLayout*** is structured with two rows:

- Top Label: displays the name of the game, "**Cannon Game**", with a custom font.
- Bottom Layout: utilizes a FloatLayout for positioning buttons vertically, which allows the player to navigate between the different sections of the game. The bottoms available are:
    - **PLAY**: takes the player to the game start screen.
    - **LEADERBOARD**: displays the leaderboard screen with the top scores.
    - **HELP**: opens a help screen with game instructions.
    - **QUIT**: exits the game.

The menu also features a custom background image, enhancing the visual presentation.

### 5.1.2 Play Button

The "**PLAY**" **button** is the first one on the menu, providing access to the save and load screen where the player can either start a new game or resume a previously saved one.

Upon pressing the "PLAY" button, the `goto_screen("save_and_load_page")` function is called, which changes the current screen to the save and load screen.

### 5.1.3 Leaderboard, Help, Quit

- **Leaderboard Button**: The leaderboard button opens a new screen displaying player rankings for each level. This is implemented via the `goto_screen("leaderboard_page")`, which manages the scores.
- **Help Button**: The help button navigates to the help screen, providing detailed information about the game mechanics, controls, and objectives. It calls the `goto_screen("help_page")` function.
- **Quit Button**: This button allows the player to exit the game by calling the `app.close()` function, which stops the running Kivy application.

Each button is designed with a custom font, background color, and size for consistent aesthetics and functionality, creating a cohesive and intuitive user experience.

## 5.2 Save and Load screen

The **Save and Load Screen** provides an interface for the player to either begin a new game by entering a nickname or resume an existing game by re-entering the same player's nickname used previously. If the nickname matches one

already saved, the game will load the corresponding saved data and continue from where the player left off.

### 5.2.1 Kivy design

The design of the insertion screen utilizes a ***FloatLayout*** to position elements freely across the screen. The key components of this screen include:

- A ***Label*** at the top with the title "Start New Game or Load Game" in custom fonts.
- A ***TextInput*** widget that allows the player to input their nickname.
- A second **Label** that displays the player's name dynamically based on the input.
- A **Submit** button, which triggers the process of saving or loading the player's game.
- A **Back** button that returns the player to the home screen.

### 5.2.3 Nickname Input and Submission

Upon entering a nickname in the ***TextInput*** field and pressing the "**Submit**" button, the `on_submit()` function is invoked. The following actions occur:

1. Load Existing Game:
    - The game checks if the nickname exists in the `record.json` file.
    - If a match is found, the game loads the corresponding saved data (score and last level) and sets it to the `player` object.
2. Start New Game:
    - If the nickname does not exist, the game creates a new entry with default values (score: 0, last level: 1).

- This new player data is appended to the JSON file, allowing future resumption by entering the same nickname.

```
        if not user_exists:
            new_user = {
                "nickname": nickname,
                "score": 0,
                "last_level": 1
            }
            data.append(new_user)
            self.manager.player = new_user
```

This mechanism provides seamless continuity for players, allowing them to pick up from where they left off by simply re-entering their previously used nickname.

## 5.3 Selection screen

### 5.3.1 Kivy design

The selection screen uses a combination of ***AnchorLayout*** and ***GridLayout*** to arrange the interface elements. Key components of this screen include:

- A **Label** displaying "PERSONALIZE YOUR TANK" at the top, styled with a custom font and sized to stand out.
- Two interactive sections where players can select both the tank model and the cannon.

- **Image widgets** display the current selection for both tank models and cannons, with buttons to navigate left and right through the available options.
- A **Select** button that finalizes the customization and saves the selected model and cannon before transitioning to the next gameplay screen.
- A **Back** button at the bottom to return to the home screen.

### 5.3.2 Tank and Cannon Selection

The selection screen allows the player to choose from a variety of tank models and cannons. The available models and cannons are defined as lists within the screen logic (***tanks*** and ***cannons*** respectively). The player can cycle through these options using buttons on either side of the images.

When the **Select** button is pressed, the `set_model()` function is triggered. This function saves the current tank and cannon selection, passing the data to the screen manager for use in subsequent levels.

The `change_path_tank()` and `change_path()` methods handle the logic for updating the tank and cannon images when the player navigates through the options. If the player presses the left or right buttons, the image updates to display the next or previous option in the list, respectively.

This system allows for an interactive and engaging selection process, giving the player control over their tank's appearance.

## 5.4 Levels screen

The **Levels Screen** is designed to allow players to select the level they want to play, serving as a central hub for level selection and progression. The layout

employs Kivy's ***AnchorLayout*** and ***GridLayout*** to create an intuitive and user-friendly interface.

### 5.4.1 Kivy design

The layout of the Levels Screen is structured as follows:

- **AnchorLayout**: This layout is utilized to center the ***GridLayout*** within the screen, providing a balanced appearance regardless of the screen size.
- **GridLayout**: The ***GridLayout*** contains buttons for each level (Level 1 to Level 5). This grid arrangement allows for easy navigation and selection, ensuring that players can quickly locate their desired level.
- **ScrollView**: A ***ScrollView*** is implemented to accommodate the buttons for all levels, especially useful as additional levels are added in future updates. This feature keeps the screen organized and prevents overcrowding.

### 5.4.2 Levels specifications

Each level is represented by a button within the ***GridLayout***. The functionality of these buttons includes:

- **Button Actions**: When a player clicks on a level button, the `on_release` action triggers the `check_level()` method to verify access.
- `check_level()` **method**: This method checks the player's *last_level* attribute to determine if the selected level is unlocked. If the player has completed the required previous levels, access to the selected level is granted.

```
    def check_level(self, n):
```

```
        if self.manager.player['last_level'] >= n:
            self.manager.goto_screen(f"level{n}_page")
        else:
            popup = NoAccess()
            popup.open()
```

- **No Access Popup**: If the player attempts to select a locked level, a `NoAccess` popup is displayed. This message informs the player that they need to complete earlier levels to unlock the selected one, thus encouraging progression through the game.

To enhance the user experience, the Levels Screen features:

- **Background Image**: A visually appealing background image sets the tone for the levels, making the interface more engaging for players.
- **Back Button**: A "Back" button is included to allow players to return to the home screen easily, ensuring smooth navigation within the game.

### 5.4.3 Level 1 as example

To explain how a level in the game is formed, let's use **Level 1** as an example and incorporate key elements like the **Tank** and **Part_tank** classes that were discussed earlier.

#### 5.4.3.1 Kivy design

Each level in the game is structured using Kivy's widget-based design, which provides a visual layout as well as the logic for gameplay dynamics. A level is

composed of various widgets representing backgrounds, terrains, characters, and interactive elements like the enemies.

For instance, in **Level 1**, we start by defining the Kivy layout in the *.kv* file:

- **Background1**: This widget defines the background image for the level.
- **Terrain**: It sets the ground area where the tank is positioned.
- **Tank** and **Part_tank**: these elements define the player's tank model and its movement/interaction logic. *Part_tank* controls the cannon angle, shooting projectiles (bullets, lasers, bombshells), and keeping track of available ammunition.

Additionally, labels are used to display critical game information like the score, bullet types, and remaining ammunition.

#### 5.4.3.2 Gameplay mechanics

The `level1` class extends the main `levels` class and adds the necessary widgets, such as *Background1*, *Terrain*, and the tank parts. The `spawn()` method places enemy units at predetermined positions when the level starts, and they are dynamically added to the screen using the `add_widget` function.

```
    def __init__(self, **kwargs):

        super().__init__(**kwargs)

        Clock.schedule_once(self.spawn)

        Clock.schedule_interval(self.update, self.time_step)
```

In addition to the basic structure, **Level 1** also contains buttons for navigating back to the levels page, displaying help information, and updating the score and available ammunition through *Label* widgets.

In conclusion, each level, like **Level 1**, combines visual elements and gameplay logic by using Kivy's widget framework. The integration of the **Tank** and **Part_tank** classes brings interactive and dynamic gameplay, with players controlling the tank's cannon to shoot projectiles at enemies. The spawning of enemies and the dynamic labels for the score and ammo further enhance the level's interactive environment.

## 5.5 Leaderboard

The **Leaderboard Screen** allows players to view the top scores in the game, sorted by player performance. This feature encourages competition and offers players a way to track their progress against others.

### 5.5.1 Kivy design

The leaderboard screen uses a ***FloatLayout*** with a background image, creating a clean and immersive environment for displaying the top players. The screen layout consists of:

- A **Label** at the top that displays the title "LEADERBOARD," styled with font and a distinct color to stand out.
- A **Back** button in the bottom-left corner, allowing the player to return to the home screen.

A **ScrollView** is centered on the screen to display the list of players in a vertically scrollable manner. This ensures that even if the leaderboard contains many players, it will remain accessible without overcrowding the screen. The leaderboard entries are displayed in a ***BoxLayout***, dynamically filled with rows of player names and their respective scores.

### 5.5.2 Player data loading and Display

The `load_leaderboard_data()` method is responsible for loading and displaying the leaderboard data. It reads from a `record.json` file that stores player data, including nicknames and scores. The player data is sorted in descending order of scores, ensuring the highest-scoring player appears at the top.

Each player's data is displayed in a ***GridLayout*** with two columns: one for the player's nickname and one for their score. The top player's entry is highlighted in gold to signify their position as the highest scorer.

To enhance the visual aesthetics, background color is added to the nickname and score labels. The use of black with 50% opacity provides contrast against the bright text, making the leaderboard easy to read while maintaining a polished look.

### 5.4.3 Dynamic Updates

The leaderboard is dynamically updated whenever the player navigates to the screen. This ensures that any new scores added during gameplay are immediately reflected on the leaderboard.

The method `on_pre_enter` is used to trigger the leaderboard data loading right before the player enters the screen. This guarantees that the displayed data is up-to-date, without requiring a manual refresh.

The `update_rect()` function ensures that the background color for the labels adjusts to their size and position, maintaining proper alignment even if the layout changes.

## 5.6 Help

The **Help Screen** provides players with essential information about the game, including instructions and tips to enhance their gameplay experience. This feature is crucial for new players to understand the game mechanics and controls.

### 5.6.1 Kivy design

The help screen is designed to present textual information clearly and accessibly. It features:

- A **Label** at the top that displays the title "HELPING YOU" (not explicitly shown in your code but typically included for consistency).
- A **Back** button at the bottom left, allowing players to return to the previous screen.

The main content area displays help text loaded from an external file, ensuring that the information is dynamic and can be easily updated without modifying the code.

```
    def load_help_text(self):

        with open("assets\\help_text\\Help_schermataHome.txt", "r") as
file:

            contenuto = file.read()

        self.ids.text_help.text = contenuto
```

### 5.5.2 Loading Help Text

The `load_help_text()` method reads from a text file. This file contains the help information that players will see when they enter the screen.

Upon entering the screen, the `on_enter()` method is triggered, calling `load_help_tex()` to fetch and display the help content. This ensures that the help text is always up-to-date whenever the player accesses the help screen.

### 5.5.3 Text Formatting

The loaded help text is displayed in a ***Label*** widget identified by *text_help*. The text is formatted with:

- A custom font is used for better readability and aesthetic appeal.
- A font size of 15dp to ensure the text is legible without overwhelming the player.
- The text color is set to black (rgb: 0, 0, 0), providing high contrast against the background for improved visibility.
- Horizontal and vertical alignment properties (*halign* and *valign*) are set to *left* and *top*, respectively, ensuring the text starts from the top left of the label, enhancing the layout's structure.

### 5.5.4 User Experience

This design not only ensures that the help text is displayed clearly but also allows for easy updates to the content as needed, making it a flexible and user-friendly feature of the game. Players can quickly refer to the help screen for guidance, enhancing their overall gaming experience.

## 6. Possible Improvements

This game is nearly complete, but there are several potential enhancements we could explore, such as:

- Adding new levels to expand the game's storyline and lore.
- Enhancing the save and load page by including buttons for each player who has previously played the game.
- Introducing new blocks and projectiles, such as blocks that can attract projectiles or absorb bomb explosions. New types of projectiles could include explosives with a trigger button or controlled by player input.
- Adding interactive elements like portals or buttons to open doors or remove otherwise unbreakable obstacles.
- Improving the game's visual appeal with upgraded graphics and animations.
- Implementing additional power-ups for the player to collect and use during gameplay.

# 7. Tools and Resources

## 7.1 Libraries used

- Numpy
- Matplotlib
- random
- from the Kivy Framework
- JSON

## 7.2 References

- Kivy Documentation: Welcome to Kivy — Kivy 2.3.0 documentation
- JSON Documentation: json — JSON encoder and decoder — Python 3.12.7 documentation